

# PRML Spring 2026 Homework 3: Recurrent Neural Networks

Yifan Ding

Code: <https://github.com/douhenhuidyf/PRML-Spring-2026/tree/main/hw3>

Email: 23373493@buaa.edu.cn

April 25, 2026

## 1 Introduction

### 1.1 Problem Statement

Given the Air Quality dataset[1], which reports on the weather and the level of pollution each hour for five years, we aim to forecast the PM2.5 concentration at the next hour based on the weather conditions and pollution levels from prior hours. The dataset includes features such as date-time, dew point, temperature, pressure, wind direction, wind speed, and cumulative hours of snow and rain. By leveraging these features, we can build a predictive model to estimate future pollution levels effectively.

### 1.2 Problem Analysis

This is a typical time series forecasting problem, where the goal is to predict future values based on historical data. The dataset contains multiple features that can influence the target variable (PM2.5 concentration), making it a multivariate time series forecasting problem. To address this, we can utilize Recurrent Neural Networks (RNNs) and their variants which are well-suited for capturing temporal dependencies in sequential data.

### 1.3 Structure of the Paper

The rest of this report is organized as follows. In Section 2, we introduce the evaluation metric (RMSE) and the recurrent models used in this homework (LSTM, stacked LSTM, and GRU). In Section 3, we describe the experimental setup, report quantitative results under different window sizes, and analyze the empirical findings. Finally, we summarize the conclusions in Section 4.

## 2 Methodology

### 2.1 Metrics

To evaluate the forecasting performance, we use the Root Mean Squared Error (RMSE), defined as:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (1)$$

where  $y_i$  is the ground-truth PM2.5 concentration and  $\hat{y}_i$  is the predicted value. RMSE penalizes large errors more heavily than absolute-error metrics and is a standard choice for regression and time-series forecasting.

## 2.2 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a class of neural networks designed to handle sequential data.

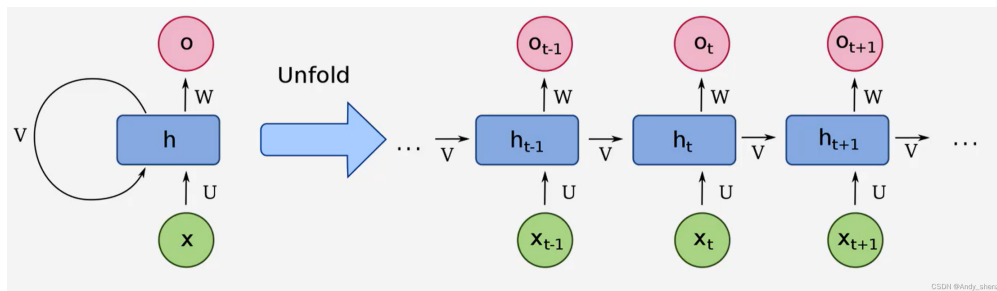


Figure 1: Architecture of a vanilla Recurrent Neural Network (RNN).

As shown in fig. 1, vanilla RNNs have a unique sequential architecture that composed of multiple identical layers of recurrent units(RNN cells). They input data in a sequential manner and maintain a hidden state that captures information from previous time steps, endowing them suitable for tasks like time series forecasting.

However, vanilla RNNs can suffer from issues like vanishing and exploding gradients since all information is passed through the hidden state. The longer the sequence, the more difficult it becomes for the model to learn long-term dependencies, which is a common problem in time series data. Besides, vanilla RNNs can be computationally expensive and time-consuming as they need to forward sequentially and backpropagating through time(BPTT). Thus vanilla RNNs can not scale up to long sequences, and we need to use more advanced architectures like Long Short Term Memory (LSTM)[3] and Gated Recurrent Units (GRU)[2] to address these issues.

## 2.3 Long Short Term Memory

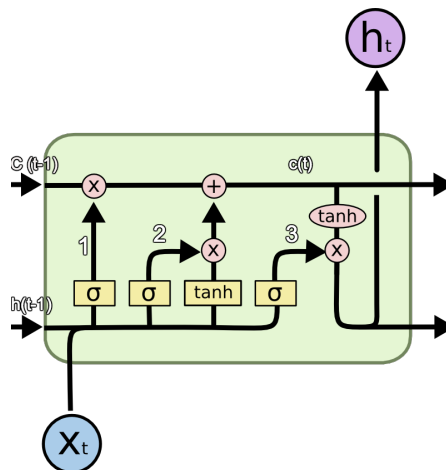


Figure 2: Architecture of a LSTM cell.

Long Short Term Memory (LSTM) is a type of RNN architecture that is designed to overcome the vanishing gradient problem in vanilla RNNs. LSTMs introduce a memory cell that can maintain information over long periods of time, allowing them to capture long-term dependencies in sequential data. The architecture of an LSTM cell is shown in fig. 2. Given input  $x_t$  and previous hidden state  $h_{t-1}$ , a standard

LSTM update can be written as

$$\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i), \\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o), \\
\tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\
h_t &= o_t \odot \tanh(c_t),
\end{aligned} \tag{2}$$

where  $\sigma(\cdot)$  is the sigmoid function and  $\odot$  denotes element-wise multiplication. LSTM uses three gates (forget gate  $f_t$ , input gate  $i_t$ , and output gate  $o_t$ ) to control the flow of information into and out of the memory cell, allowing it to retain relevant information and discard irrelevant information effectively.

## 2.4 Gated Recurrent Units

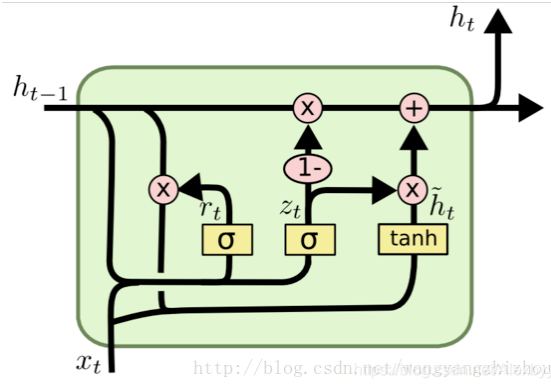


Figure 3: Architecture of a GRU cell.

Gated Recurrent Units (GRU) are another gated RNN variant proposed to mitigate vanishing gradients while keeping a relatively simple structure[2]. Compared with LSTM, GRU merges the cell state and hidden state into a single hidden representation and uses two gates: an update gate  $z_t$  and a reset gate  $r_t$ . The architecture of an GRU cell is shown in fig. 3. Given input  $x_t$  and previous hidden state  $h_{t-1}$ , a standard GRU update can be written as

$$\begin{aligned}
z_t &= \sigma(W_z[h_{t-1}, x_t] + b_z), \\
r_t &= \sigma(W_r[h_{t-1}, x_t] + b_r), \\
\tilde{h}_t &= \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h), \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t,
\end{aligned} \tag{3}$$

Empirically, GRUs often achieve performance comparable to LSTMs with fewer parameters, which can be advantageous when the dataset size is limited.

## 2.5 Stacked LSTM (Multi-layer LSTM)

In addition to a single-layer LSTM, we also evaluate a stacked LSTM (multi-layer LSTM), which composes multiple recurrent layers. At each time step, the hidden sequence produced by a lower layer is fed into the next layer. This increases model capacity and allows the network to learn hierarchical temporal representations. In our implementation, we stack LSTM layers and use the top-layer hidden state at the last time step as the representation for predicting the next-hour PM2.5 value.

## 3 Experiments

### 3.1 Experiment Setup

We formulate one-step-ahead forecasting: given a window of past observations with length  $w$ , the model predicts the PM2.5 concentration at the next hour. Concretely, each training sample is a sequence  $X_{t-w+1:t} \in \mathbb{R}^{w \times d}$  and the label is  $y_{t+1}$ . The performance is evaluated for  $w \in \{1, 3, 6, 12, 24, 36\}$  and the test RMSE is reported. The 8-dimensional normalized input feature vector per hour includes: PM2.5 (pollution), dew point (dew), temperature (temp), pressure (press), encoded wind direction (wnd\_dir), wind speed (wnd\_spd), snow, and rain. Wind direction is mapped from strings to numeric indices.

We compare three self-implemented recurrent models:

- GRU: a GRU-based regressor implemented from scratch.
- LSTM: a single-layer LSTM implemented from scratch.
- Stacked LSTM: a multi-layer LSTM implemented from scratch.

All models predict a scalar output through a linear layer applied to the final hidden state. Beside, these models are initialized orthogonally and optimized in the mean squared error (MSE) on the standardized target using Adam. We use batch size 256 and train for 100 epochs and an exponential learning-rate decay with initial learning rate  $10^{-3}$  is applied during training.

### 3.2 Results

Using the test RMSE as the evaluation metric, the results under different window sizes and recurrent architectures are shown in Table 1.

Table 1: Test RMSE under different window sizes  $w$  and recurrent architectures.

| Window size $w$ | GRU     | LSTM    | Stacked LSTM |
|-----------------|---------|---------|--------------|
| 1               | 26.5417 | 26.1787 | 26.2735      |
| 3               | 24.3073 | 24.3970 | 22.7513      |
| 6               | 18.4879 | 17.3723 | 9.2516       |
| 12              | 13.2444 | 11.1432 | 4.7362       |
| 24              | 8.1167  | 8.9399  | 4.4848       |
| 36              | 9.3022  | 10.2147 | 4.8785       |

The training loss curves (MSE on standardized target) for different window sizes are shown in Figure 4.

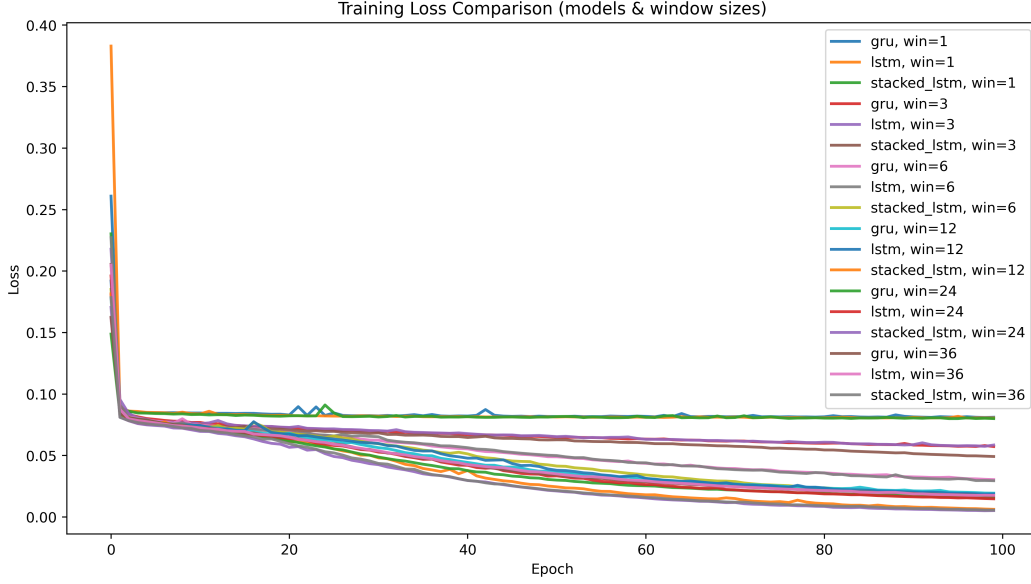


Figure 4: Training loss curves (MSE on standardized target) for different window sizes.

In Table 1, increasing the window size generally improves forecasting accuracy. When  $w$  is small (e.g., 1 or 3), the model observes only very limited temporal context and all three architectures perform similarly (RMSE around 24–26). As  $w$  increases to 6 and 12, the benefit of longer temporal context becomes clear and RMSE drops substantially. For the stacked LSTM, the RMSE decreases from 26.2735 ( $w = 1$ ) to 4.4848 ( $w = 24$ ). When the window becomes too large (e.g.,  $w = 36$ ), performance no longer improves and can slightly degrade, which may be caused by optimization difficulty and noisy long-range inputs.

The stacked LSTM consistently outperforms the single-layer LSTM and GRU once the window size is sufficiently large. For instance, at  $w = 12$ , stacked LSTM achieves RMSE 4.7362, significantly better than LSTM (11.1432) and GRU (13.2444). This suggests that increasing depth helps the model extract higher-level temporal patterns from longer contexts. Between GRU and LSTM, their performance is comparable at small windows, and neither dominates across all settings.

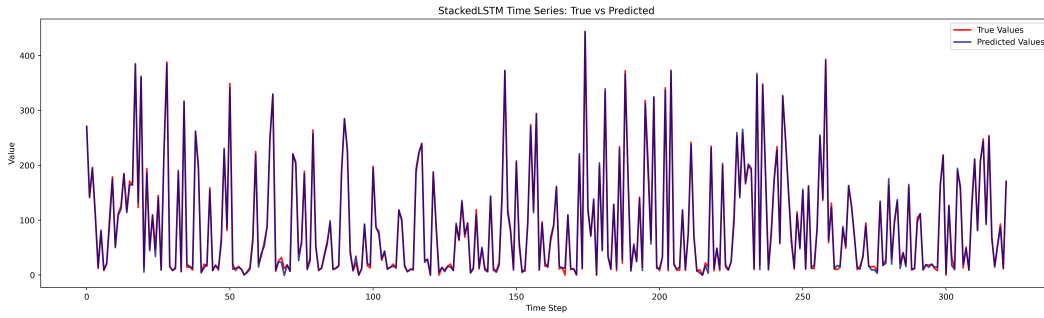


Figure 5: True vs. Predicted Values for Stacked LSTM with Window Size  $w = 24$ .

Among all settings tested, the best RMSE is achieved by stacked LSTM with  $w = 24$  (RMSE 4.4848). As shown in Figure 5, the predicted PM2.5 values closely follow the true values, indicating strong forecasting performance. This highlights the importance of both sufficient temporal context and model capacity for accurate next-step pollution forecasting.

## 4 Conclusions

We studied multivariate time-series forecasting on an hourly air-quality dataset using recurrent neural networks. All models were implemented from scratch and evaluated under different historical window sizes. Using RMSE as the metric, we observe that larger windows generally improve accuracy up to a point, and stacked (multi-layer) LSTM provides the best performance when sufficient temporal context is available. The best configuration in our experiments is the stacked LSTM with window size  $w = 24$ , achieving a test RMSE of 4.4848. Overall, these results highlight that both temporal context length and model capacity are crucial for improving next-step pollution forecasting.

## References

- [1] Rupak Roy/ Bob. *Air Pollution Forecasting - LSTM Multivariate*. <https://www.kaggle.com/datasets/rupakroy/lstm-datasets-multivariate-univariate>. 2022.
- [2] Kyunghyun Cho et al. *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. 2014. arXiv: 1406.1078 [cs.CL]. URL: <https://arxiv.org/abs/1406.1078>.
- [3] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.